

TOPIC 4 : DYNAMIC PROGRAMMING

1. You are given the number of sides on a die (num_sides), the number of dice to throw (num_dice), and a target sum (target). Develop a program that utilizes dynamic programming to solve the Dice Throw Problem.

Test Cases:

1.Simple Case:

- Number of sides: 6

- Number of dice: 2

- Target sum: 7

2.More Complex Case:

- Number of sides: 4

- Number of dice: 3

- Target sum: 10

Output

Test Case 1:

Number of ways to reach sum 7: 6

Test Case 2:

Number of ways to reach sum 10: 27

Aim

To find the number of ways to get a target sum using given dice and sides using Dynamic Programming.

Algorithm

1. Create a DP table for dice and sums.
2. Initialize base case $dp[0][0] = 1$.
3. For each dice, calculate possible sums.
4. Add ways from previous states.
5. Return number of ways for target sum.

Python Code

```
def dice(num_dice,num_sides,target):  
  
    dp=[[0]*(target+1) for _ in range(num_dice+1)]  
  
    dp[0][0]=1  
  
    for i in range(1,num_dice+1):  
        for j in range(1,target+1):  
            for k in range(1,num_sides+1):  
                if j-k>=0:  
                    dp[i][j]+=dp[i-1][j-k]  
  
    return dp[num_dice][target]  
  
print("Ways =",dice(2,6,7))  
  
print("Ways =",dice(3,4,10))
```

Input

Number of sides = 6
Number of dice = 2
Target = 7

Number of sides = 4
Number of dice = 3
Target = 10

Output

```
type Copyright , credits or license() for more information.
>>> ===== RESTART =====
>>>
('Ways =', 6)
('Ways =', 6)
>>>
```

Result

The program finds the number of possible ways using Dynamic Programming.
Time complexity is $O(\text{dice} \times \text{target} \times \text{sides})$.

2. In a factory, there are two assembly lines, each with n stations. Each station performs a specific task and takes a certain amount of time to complete. The task must go through each station in order, and there is also a transfer time for switching from one line to another. Given the time taken at each station on both lines and the transfer time between the lines, the goal is to find the minimum time required to process a product from start to end.

Input

n: Number of stations on each line.

a1[i]: Time taken at station i on assembly line 1.

a2[i]: Time taken at station i on assembly line 2.

t1[i]: Transfer time from assembly line 1 to assembly line 2 after station i .

t2[i]: Transfer time from assembly line 2 to assembly line 1 after station i .

e1: Entry time to assembly line 1.

e2: Entry time to assembly line 2.

x1: Exit time from assembly line 1.

x2: Exit time from assembly line 2.

Output

The minimum time required to process the product.

Aim

To find the minimum processing time in assembly line scheduling using Dynamic Programming.

Algorithm

1. Compute minimum time for each station on both lines.
2. Compare staying on same line or switching lines.
3. Store minimum values dynamically.
4. Add exit times and return minimum result.

Python Code

```
File Edit Format Run Options Windows Help
a1=[4,5,3,2]
a2=[2,10,1,4]

t1=[7,4,5]
t2=[9,2,8]

e1,e2=10,12
x1,x2=18,7

n=len(a1)

f1=[0]*n
f2=[0]*n

f1[0]=e1+a1[0]
f2[0]=e2+a2[0]

for i in range(1,n):

    f1[i]=min(f1[i-1]+a1[i],
              f2[i-1]+t2[i-1]+a1[i])

    f2[i]=min(f2[i-1]+a2[i],
              f1[i-1]+t1[i-1]+a2[i])

ans=min(f1[n-1]+x1,f2[n-1]+x2)

print("Minimum Time =",ans)
```

Input

```
a1 = [4,5,3,2]
a2 = [2,10,1,4]

t1 = [7,4,5]
t2 = [9,2,8]
```

Output

```
>>> ===== RESTART =====
>>>
('Minimum Time =', 35)
>>> |
```

Result

The program calculates the minimum assembly processing time efficiently.
Dynamic Programming avoids repeated calculations and improves performance.

3. An automotive company has three assembly lines (Line 1, Line 2, Line 3) to produce different car models. Each line has a series of stations, and each station takes a certain amount of time to complete its task. Additionally, there are transfer times between lines, and certain dependencies must be respected due to the sequential nature of some tasks. Your goal is to minimize the total production time by determining the optimal scheduling of tasks across these lines, considering the transfer times and dependencies.

Number of stations: 3

- Station times:
- Line 1: [5, 9, 3]
- Line 2: [6, 8, 4]
- Line 3: [7, 6, 5]
- Transfer times:

```
[
[0, 2, 3],
[2, 0, 4],
[3, 4, 0]
```

]

Dependencies: [(0, 1), (1, 2)] (i.e., the output of the first station is needed for the second, and the second for the third, regardless of the line).

Aim

To find the minimum production time across three assembly lines using Dynamic Programming.

Algorithm

1. Store minimum time for each line at every station.
2. Add station time and minimum transfer cost.
3. Compare all possible previous lines.
4. Select minimum total production time.

Python Code

```
File Edit Format Run Options Windows Help
line1=[5,9,3]
line2=[6,8,4]
line3=[7,6,5]

t=[[0,2,3],
   [2,0,4],
   [3,4,0]]

dp=[[0]*3 for _ in range(3)]

dp[0]=[5,6,7]

for i in range(1,3):

    dp[i][0]=min(dp[i-1][0]+line1[i],
                 dp[i-1][1]+t[1][0]+line1[i],
                 dp[i-1][2]+t[2][0]+line1[i])

    dp[i][1]=min(dp[i-1][1]+line2[i],
                 dp[i-1][0]+t[0][1]+line2[i],
                 dp[i-1][2]+t[2][1]+line2[i])

    dp[i][2]=min(dp[i-1][2]+line3[i],
                 dp[i-1][0]+t[0][2]+line3[i],
                 dp[i-1][1]+t[1][2]+line3[i])

print("Minimum Time =",min(dp[2]))
```

Input

Line 1 = [5,9,3]

Line 2 = [6,8,4]

Line 3 = [7,6,5]

Transfer Matrix =

[[0,2,3],

[2,0,4],

[3,4,0]]

Output

```
>>> ===== RESTART =====  
>>>  
('Minimum Time =', 17)
```

Result

The program finds the optimal assembly scheduling using Dynamic Programming.

Time complexity is $O(n \times \text{lines}^2)$.

4. Write a c program to find the minimum path distance by using matrix form. Test Cases:

1)

{0,10,15,20}

{10,0,35,25}

{15,35,0,30}

{20,25,30,0}

Output: 80

2)

{0,10,10,10}

{10,0,10,10}

{10,10,0,10}

{10,10,10,0}

Output: 40

3)

{0,1,2,3}

{1,0,4,5}

{2,4,0,6}

{3,5,6,0}

Output: 12

Aim

To find the minimum path distance using matrix representation.

Algorithm

1. Generate all possible paths.
2. Compute total distance for each path.
3. Track minimum distance.
4. Return shortest path distance.

Python Code

```
File Edit Format Run Options Windows Help
from itertools import permutations

a=[[0,10,15,20],
   [10,0,35,25],
   [15,35,0,30],
   [20,25,30,0]]

n=len(a)

cities=[1,2,3]

min_cost=999

for p in permutations(cities):

    cost=0
    k=0

    for j in p:
        cost+=a[k][j]
        k=j

    cost+=a[k][0]

    min_cost=min(min_cost,cost)

print("Minimum Distance =",min_cost)
```

Input

{0,10,15,20}
{10,0,35,25}
{15,35,0,30}
{20,25,30,0}

Output

```
>>> ===== RESTART =====
>>>
('Minimum Distance =', 80)
```

Result

The program checks all possible paths to find minimum distance.

Time complexity is $O(n!)$.

5. Assume you are solving the Traveling Salesperson Problem for 4 cities (A, B, C, D) with known distances between each pair of cities. Now, you need to add a fifth city (E) to the problem.

Test Cases

1. Symmetric Distances

- **Description:** All distances are symmetric (distance from A to B is the same as B to A).

Distances:

A-B: 10, A-C: 15, A-D: 20, A-E: 25 B-C: 35, B-D: 25, B-E: 30 C-D: 30, C-E: 20 D-E: 15

Expected Output: The shortest route and its total distance. For example, A -> B -> D -> E -> C -> A might be the shortest route depending on the given distances.

Aim

To find the shortest route that visits all cities exactly once and returns to the starting city.

Algorithm

1. Store distances between cities.
2. Generate all possible routes using permutations.
3. Calculate total distance for each route.
4. Select the route with minimum distance.

Python Code

```
File Edit Format Run Options Windows Help
from itertools import permutations

d={
('A','B'):10, ('A','C'):15, ('A','D'):20, ('A','E'):25,
('B','C'):35, ('B','D'):25, ('B','E'):30,
('C','D'):30, ('C','E'):20,
('D','E'):15
}

def dist(x,y):
    if (x,y) in d:
        return d[(x,y)]
    return d[(y,x)]

cities=['B','C','D','E']

min_cost=999
best=[]

for p in permutations(cities):

    path=['A']+list(p)+['A']

    cost=0

    for i in range(len(path)-1):
        cost+=dist(path[i],path[i+1])

    if cost<min_cost:
        min_cost=cost
        best=path

print("Shortest Route =",best)
print("Distance =",min cost)
```

Input

A-B:10, A-C:15, A-D:20, A-E:25

B-C:35, B-D:25, B-E:30

C-D:30, C-E:20

D-E:15

Output

```
>>> ===== RESTART =====  
>>>  
( 'Shortest Route =', ['A', 'B', 'D', 'E', 'C', 'A'])  
( 'Distance =', 85)  
>>>
```

Result

The program checks all possible routes to find the shortest path.

Time complexity is $O(n!)$.

6. Given a string s , return the longest palindromic substring in S . Example 1:

Input: $s = \text{"babad"}$

Output: "bab" **Explanation:** "aba" is also a valid answer.

Example 2:

Input: $s = \text{"cbbd"}$

Output: "bb"

Constraints: • $1 \leq s.length \leq 1000$ • s consist of only digits and English letters.

Aim

To find the longest palindromic substring in a given string using Dynamic Programming.

Algorithm

1. Check every substring.
2. Verify whether substring is palindrome.
3. Store the longest palindrome found.
4. Return longest palindromic substring.

Python Code

File Edit Format Run Options Windows Help

```
def longest(s):  
    ans=""  
    for i in range(len(s)):  
        for j in range(i, len(s)):  
            sub=s[i:j+1]  
            if sub==sub[::-1] and len(sub)>len(ans):  
                ans=sub  
    return ans  
print(longest("babad"))  
print(longest("cbbd"))
```

Input

s = "babad"

s = "cbbd"

Output

```
>>> ===== RESTART =====  
>>>  
bab  
bb
```

Result

The program finds the longest palindrome substring correctly.
Time complexity is $O(n^3)$.

7. Given a string s, find the length of the longest substring without repeating characters.

Example 1: Input: s = "abcabcbb" Output: 3

Explanation: The answer is "abc", with the length of 3.

Example 2: Input: s = "bbbbb" Output: 1

Explanation: The answer is "b", with the length of 1.

Example 3: Input: s = "pwwkew" Output: 3

Explanation: The answer is "wke", with the length of 3.

Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

Constraints: • $0 \leq s.length \leq 5 * 10^4$ • s consists of English letters, digits, symbols and spaces.

Aim

To find the length of the longest substring without repeating characters.

Algorithm

1. Use a sliding window technique.
2. Store visited characters in a set.
3. Remove repeated characters from window.
4. Track maximum substring length.

Python Code

File Edit Format Run Options Windows Help

```
def longest(s):  
    seen=set()  
    l=0  
    ans=0  
    for r in range(len(s)):  
        while s[r] in seen:  
            seen.remove(s[l])  
            l+=1  
        seen.add(s[r])  
        ans=max(ans,r-l+1)  
    return ans  
print(longest("abcabcbb"))  
print(longest("bbbbbb"))  
print(longest("pwwkew"))
```

Input

s = "abcabcbb"

s = "bbbbbb"

s = "pwwkew"

Output

```
>>> ===== RESTART =====  
>>>  
3  
1  
3
```

Result

The program finds the longest substring without repeating characters efficiently.
Time complexity is $O(n)$.

8. Given a string s and a dictionary of strings $wordDict$, return true if s can be segmented into a space-separated sequence of one or more dictionary words.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

Input: $s = \text{"leetcode"}$, $wordDict = [\text{"leet"}, \text{"code"}]$

Output: true

Explanation: Return true because "leetcode" can be segmented as "leet code".

Example 2:

Input: $s = \text{"applepenapple"}$, $wordDict = [\text{"apple"}, \text{"pen"}]$

Output: true

Explanation: Return true because "applepenapple" can be segmented as "apple pen apple".

Note that you are allowed to reuse a dictionary word.

Example 3:

Input: $s = \text{"catsanddog"}$, $wordDict = [\text{"cats"}, \text{"dog"}, \text{"sand"}, \text{"and"}, \text{"cat"}]$

Output: false

Aim

To check whether a string can be segmented into valid dictionary words using Dynamic Programming.

Algorithm

1. Create a DP array of size $n+1$.
2. Mark $dp[0]$ as True.
3. Check all substrings in dictionary.
4. Update DP values if segmentation is possible.
5. Return final DP result.

Python Code

```
File Edit Format Run Options Windows Help
def wordBreak(s,wordDict):
    dp=[False]*(len(s)+1)
    dp[0]=True
    for i in range(1,len(s)+1):
        for j in range(i):
            if dp[j] and s[j:i] in wordDict:
                dp[i]=True
    return dp[len(s)]

print(wordBreak("leetcode",["leet","code"]))

print(wordBreak("applepenapple",["apple","pen"]))

print(wordBreak("catsandog",
["cats","dog","sand","and","cat"]))
```

Input

s = "leetcode"

wordDict = ["leet","code"]

s = "applepenapple"

wordDict = ["apple","pen"]

s = "catsandog"

wordDict = ["cats","dog","sand","and","cat"]

Output

```
>>> ===== RESTART =====
>>>
True
True
False
>>>
```

Result

The program checks whether the string can be segmented into dictionary words.

Time complexity is $O(n^2)$.

9. Given an input string and a dictionary of words, find out if the input string can be segmented into a space-separated sequence of dictionary words. Consider the following dictionary { i, like, sam, sung, samsung, mobile, ice, cream, icecream, man, go, mango }

Input: ilike

Output: Yes

The string can be segmented as "i like".

Input: ilikesamsung

Output: Yes The string can be segmented as "i like samsung" or "i like sam sung".

Aim

To check whether the input string can be segmented into valid dictionary words.

Algorithm

1. Create a DP array.
2. Mark $dp[0]$ as True.
3. Check all substrings with dictionary words.
4. Update DP values if valid segmentation exists.
5. Return final result.

Python Code

```
File Edit Format Run Options Windows Help
def wordBreak(s, wordDict):

    dp=[False]*(len(s)+1)

    dp[0]=True

    for i in range(1, len(s)+1):

        for j in range(i):

            if dp[j] and s[j:i] in wordDict:
                dp[i]=True

    return dp[len(s)]

d=["i", "like", "sam", "sung", "samsung",
  "mobile", "ice", "cream", "icecream",
  "man", "go", "mango"]

print(wordBreak("ilike", d))

print(wordBreak("ilikesamsung", d))
```

Input

s = "ilike"

s = "ilikesamsung"

Dictionary = {i, like, sam, sung, samsung, mobile, ice, cream, icecream, man, go, mango}

Output

```
>>> ===== RESTART =====
>>>
True
True
```

Result

The program checks whether the string can be segmented into dictionary words.

Time complexity is $O(n^2)$.

10. Given an array of strings `words` and a width `maxWidth`, format the text such that each line has exactly `maxWidth` characters and is fully (left and right) justified. You should pack your words in a greedy approach; that is, pack as many words as you can in each line. Pad extra spaces ' ' when necessary so that each line has exactly `maxWidth` characters. Extra spaces between words should be distributed as evenly as possible. If the number of spaces on a line does not divide evenly between words, the empty slots on the left will be assigned more spaces than the slots on the right. For the last line of text, it should be left-justified, and no extra space is inserted between words. A word is defined as a character sequence consisting of non-space characters only. Each word's length is guaranteed to be greater than 0 and not exceed `maxWidth`. The input array `words` contains at least one word.

Example 1:

Input: `words = ["This", "is", "an", "example", "of", "text", "justification."],`
`maxWidth = 16`

Output:

```
[ "This is an",  
  "example of text",  
  "justification. "  
]
```

Example 2:

Input: `words = ["What","must","be","acknowledgment","shall","be"],`
`maxWidth = 16`

Output:

```
[  
  "What must be",  
  "acknowledgment ",  
  "shall be "  
]
```

Explanation: Note that the last line is "shall be " instead of "shall be", because the last line must be left-justified instead of fully-justified.

Note that the second line is also left-justified because it contains only one word.

Aim

To format text so that each line has equal width and is properly justified.

Algorithm

1. Add maximum words possible in one line.
2. Calculate remaining spaces.
3. Distribute spaces evenly between words.
4. Left justify the last line.

Python Code

```
File Edit Format Run Options Windows Help
def justify(words,maxWidth):

    res=[]
    line=[]

    length=0

    for w in words:

        if length+len(w)+len(line)>maxWidth:

            spaces=maxWidth-length

            for i in range(spaces):

                line[i%(len(line)-1 or 1)] += ' '

            res.append(' '.join(line))

            line=[]
            length=0

        line.append(w)
        length+=len(w)

    res.append(' '.join(line).ljust(maxWidth))

    return res

words=["This","is","an","example",
       "of","text","justification."]

print(justify(words,16))
```

Input

```
words = ["This","is","an","example","of","text","justification."]
```

```
maxWidth = 16
```

Output

```
>>> ===== RESTART =====
>>>
['This    is    an', 'example of text', 'justification. ']
```

Result

The program formats text with proper spacing and justification.

Time complexity is $O(n)$.

11. Design a special dictionary that searches the words in it by a prefix and a suffix. Implement the **WordFilter** class: **WordFilter(string[] words)** Initializes the object with the words in the dictionary.**f(string pref, string suff)** Returns the index of the word in the dictionary, which has the prefix **pref** and the suffix **suff**. If there is more than one valid index, return the largest of them. If there is no such word in the dictionary, return -1.

Example 1:

Input

```
["WordFilter", "f"]
```

```
[["apple"], ["a", "e"]]
```

Output

```
[null, 0]
```

Explanation

```
WordFilter wordFilter = new WordFilter(["apple"]);
```

```
wordFilter.f("a", "e"); // return 0, because the word at index 0 has prefix = "a" and
suffix = "e".
```

Aim

To design a dictionary that searches words using both prefix and suffix.

Algorithm

1. Store all words with their indices.
2. Check each word for matching prefix and suffix.
3. Return the largest valid index.
4. Return -1 if no match exists.

Python Code

```
class WordFilter:

    def __init__(self, words):
        self.words=words

    def f(self, pref, suff):

        ans=-1

        for i,w in enumerate(self.words):

            if w.startswith(pref) and w.endswith(suff):
                ans=i

        return ans

wf=WordFilter(["apple"])

print(wf.f("a","e"))
```

Input

words = ["apple"]

Prefix = "a"

Suffix = "e"

Output

```
>>> ===== RESTART =====
>>>
0
>>>
```

Result

The program searches words using prefix and suffix conditions.
Time complexity is $O(n \times \text{length of word})$.

12. Implement Floyd's Algorithm to find the shortest path between all pairs of cities. Display the distance matrix before and after applying the algorithm. Identify and print the shortest path

Input: $n = 4$, edges = $[[0,1,3],[1,2,1],[1,3,4],[2,3,1]]$, distanceThreshold = 4

Output: 3

Explanation: The figure above describes the graph.

The neighboring cities at a distanceThreshold = 4 for each city are: City

0 -> [City 1, City 2]

City 1 -> [City 0, City 2, City 3]

City 2 -> [City 0, City 1, City 3]

City 3 -> [City 1, City 2]

Cities 0 and 3 have 2 neighboring cities at a distanceThreshold = 4, but we have to return city 3 since it has the greatest number.

Test cases :

a) You are given a small network of 4 cities connected by roads with the following distances:

City 1 to City 2: 3

City 1 to City 3: 8

City 1 to City 4: -4

City 2 to City 4: 1

City 2 to City 3: 4

City 3 to City 1: 2

City 4 to City 3: -5

City 4 to City 2: 6

Implement Floyd's Algorithm to find the shortest path between all pairs of cities. Display the distance matrix before and after applying the algorithm. Identify and print the shortest path from City 1 to City 3.

Input as above

Output : City 1 to City 3 = -9

b. Consider a network with 6 routers. The initial routing table is as follows:

Router A to Router B: 1

Router A to Router C: 5

Router B to Router C: 2

Router B to Router D: 1

Router C to Router E: 3

Router D to Router E: 1

Router D to Router F: 6

Router E to Router F: 2

Aim

To find shortest paths between all pairs of vertices using Floyd Warshall Algorithm.

Algorithm

1. Store graph in matrix form.
2. Use each vertex as intermediate node.
3. Update shortest distances dynamically.
4. Print final shortest path matrix.

Python Code

```
File Edit Format Run Options Windows Help
INF=999

g=[[0,3,INF,-4],
   [INF,0,4,1],
   [2,INF,0,INF],
   [INF,6,-5,0]]

n=4

print("Before:")

for i in g:
    print(i)

for k in range(n):

    for i in range(n):

        for j in range(n):

            g[i][j]=min(g[i][j],
                        g[i][k]+g[k][j])

print("\nAfter:")

for i in g:
    print(i)

print("\nCity 1 to City 3 =",g[0][2])
|
```

Input

City1→City2 = 3
City1→City4 = -4
City2→City3 = 4
City2→City4 = 1
City3→City1 = 2
City4→City2 = 6
City4→City3 = -5

Output

```
>>> ===== KESIAKI =====
>>>
Before:
[0, 3, 999, -4]
[999, 0, 4, 1]
[2, 999, 0, 999]
[999, 6, -5, 0]

After:
[-7, -4, -9, -11]
[-2, 0, -4, -6]
[-5, -2, -7, -9]
[-10, -7, -12, -14]
('\nCity 1 to City 3 =', -9)
```

Result

The program computes shortest paths between all city pairs efficiently.
Time complexity is $O(n^3)$.

13. Write a Program to implement Floyd's Algorithm to calculate the shortest paths between all pairs of routers. Simulate a change where the link between Router B and Router D fails. Update the distance matrix accordingly. Display the shortest path from Router A to Router F before and after the link failure.

Input as above

Output : Router A to Router F = 5

Aim

To find shortest paths between all routers using Floyd's Algorithm and update paths after a link failure.

Algorithm

1. Create distance matrix for routers.
2. Apply Floyd's Algorithm to update shortest paths.
3. Remove failed link and update matrix.
4. Recalculate shortest paths.

Python Code

```
File Edit Format Run Options Windows Help
INF = 999

routers = ["A", "B", "C", "D", "E", "F"]

dist = [
    [0, 1, 5, INF, INF, INF],
    [1, 0, 2, 1, INF, INF],
    [5, 2, 0, INF, 3, INF],
    [INF, 1, INF, 0, 1, 6],
    [INF, INF, 3, 1, 0, 2],
    [INF, INF, INF, 6, 2, 0]
]

n = len(dist)

def floyd(d):
    for k in range(n):
        for i in range(n):
            for j in range(n):
                d[i][j] = min(d[i][j], d[i][k] + d[k][j])
    return d

# Before failure
res = floyd([row[:] for row in dist])
print("Before Failure A to F =", res[0][5])

# Remove B-D link
dist[1][3] = INF
dist[3][1] = INF

# After failure
res2 = floyd([row[:] for row in dist])
print("After Failure A to F =", res2[0][5])
|
```

Input

Routers with connections:

A-B=1, A-C=5, B-C=2, B-D=1, C-E=3, D-E=1, D-F=6, E-F=2

Output

```
>>> ===== RESTART =====
>>>
('Before Failure A to F =', 5)
('After Failure A to F =', 8)
```

Result

The program finds shortest paths using Floyd's Algorithm.
After link failure, the routing path updates automatically.

14. Implement Floyd's Algorithm to find the shortest path between all pairs of cities. Display the distance matrix before and after applying the algorithm. Identify and print the shortest path

Input: n = 5, edges = [[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]], distanceThreshold = 2

Output: 0

Explanation: The figure above describes the graph.

The neighboring cities at a distanceThreshold = 2 for each city are: City

0 -> [City 1]

City 1 -> [City 0, City 4]

City 2 -> [City 3, City 4]

City 3 -> [City 2, City 4]

City 4 -> [City 1, City 2, City 3]

The city 0 has 1 neighboring city at a distanceThreshold = 2.

a) Test cases :

B to A 2

A TO C 3

C TO D 1

D TO A 6

C TO B 7

Find shortest path from C to A

Output = 7

b) Find shortest path from E to C

C TO A 2

A TO B 4

B TO C 1

B TO E 6

E TO A 1

A TO D 5

D TO E 2

E TO D 4

D TO C 1

C TO D 3

Output : E to C = 5

Aim

To find shortest paths between all cities using Floyd's Algorithm.

Algorithm

1. Create graph using adjacency matrix.
2. Apply Floyd's Algorithm.
3. Print shortest path between required cities.

Python Code

```
INF = 999

dist = [
    [0,2,3,INF],
    [2,0,INF,INF],
    [INF,7,0,1],
    [6,INF,INF,0]
]

n = len(dist)

for k in range(n):
    for i in range(n):
        for j in range(n):
            dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

print("Shortest Path C to A =", dist[2][0])
```

Input

B→A=2, A→C=3, C→D=1, D→A=6, C→B=7

Output

```
>>>
('Shortest Path C to A =', 7)
```

Result

The program calculates shortest paths between all city pairs.
Floyd's Algorithm works efficiently for weighted graphs.

14(b). Floyd's Algorithm – E to C

Aim

To find shortest path from E to C using Floyd's Algorithm.

Algorithm

1. Create adjacency matrix.
2. Apply Floyd's Algorithm.
3. Print shortest distance from E to C.

Python Code

```
INF = 999

dist = [
    [0, 4, INF, 5, INF],
    [INF, 0, 1, INF, 6],
    [2, INF, 0, 3, INF],
    [INF, INF, 1, 0, 2],
    [1, INF, INF, 4, 0]
]

n = len(dist)

for k in range(n):
    for i in range(n):
        for j in range(n):
            dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

print("E to C =", dist[4][2])
```

Input

C→A=2, A→B=4, B→C=1, B→E=6, E→A=1, A→D=5, D→E=2, E→D=4, D→C=1, C→D=3

Output

```
('E to C =', 5)
>>>
```

Result

The program finds the minimum distance between cities.

Floyd's Algorithm computes all-pairs shortest paths correctly.

15. Implement the Optimal Binary Search Tree algorithm for the keys A,B,C,D with frequencies 0.1,0.2,0.4,0.3 Write the code using any programming language to construct the OBST for the given keys and frequencies. Execute your code and display the resulting OBST and its cost. Print the cost and root matrix.

Input N =4, Keys = {A,B,C,D} Frequencies = {0.1,0.2,0.4,0.3}

Output : 1.7

Cost Table

	0	1	2	3	4
1	0	0.1	0.4	1.1	1.7
2	0	0.2	0.8	0.4	
3	0	0.4	1.0		
4	0	0.3			
5	0				

Root table

	1	2	3	4
1	1	2	3	3
2	2	2	3	3
3	3	3	3	

4 4

a) Test cases

Input: keys[] = {10, 12}, freq[] = {34, 50}

Output = 118

b) Input: keys[] = {10, 12, 20}, freq[] = {34, 8, 50}

Output = 142

Aim

To construct an Optimal Binary Search Tree using given keys and frequencies with minimum search cost.

Algorithm

1. Store keys and frequencies.
2. Create cost and root matrices.
3. Compute minimum search cost using Dynamic Programming.
4. Print cost table, root table, and minimum cost.

Python Code

```
keys = ['A', 'B', 'C', 'D']
freq = [0.1, 0.2, 0.4, 0.3]

n = len(freq)

cost = [[0]*(n+1) for _ in range(n+1)]
root = [[0]*(n+1) for _ in range(n+1)]

for i in range(n):
    cost[i][i+1] = freq[i]
    root[i][i+1] = i+1

for l in range(2, n+1):
    for i in range(n-l+1):
        j = i+l
        cost[i][j] = 999
        s = sum(freq[i:j])

        for r in range(i+1, j+1):
            c = cost[i][r-1] + cost[r][j] + s

            if c < cost[i][j]:
                cost[i][j] = c
                root[i][j] = r

print("Minimum Cost =", round(cost[0][n], 1))

print("Cost Matrix")
for row in cost:
    print(row)

print("Root Matrix")
for row in root:
    print(row)
```

Input

Keys = {A,B,C,D}

Frequencies = {0.1,0.2,0.4,0.3}

Output

```
>>> ===== RESTART =====
>>>
('Minimum Cost =', 1.7)
Cost Matrix
[0, 0.1, 0.4, 1.1, 1.7]
[0, 0, 0.2, 0.8, 1.4000000000000001]
[0, 0, 0, 0.4, 1.0]
[0, 0, 0, 0, 0.3]
[0, 0, 0, 0, 0]
Root Matrix
[0, 1, 2, 3, 3]
[0, 0, 2, 3, 3]
[0, 0, 0, 3, 3]
[0, 0, 0, 0, 4]
[0, 0, 0, 0, 0]
```

Result

The OBST minimizes search cost using Dynamic Programming.
The minimum optimal search cost obtained is 1.7.

16. Consider a set of keys 10,12,16,21 with frequencies 4,2,6,3 and the respective probabilities. Write a Program to construct an OBST in a programming language of your choice. Execute your code and display the resulting OBST, its cost and root matrix.

Input N =4, Keys = {10,12,16,21} Frequencies = {4,2,6,3}

Output : 26

0 1 2 3

0 4 80 202 262

1 2 102 162

2 6 12

3 3

a) Test cases

Input: keys[] = {10, 12}, freq[] = {34, 50}

Output = 118

b) Input: keys[] = {10, 12, 20}, freq[] = {34, 8, 50}

Output = 142

Aim

To construct an Optimal Binary Search Tree and compute minimum search cost.

Algorithm

1. Initialize cost and root matrices.
2. Fill matrices using Dynamic Programming.
3. Choose roots with minimum cost.
4. Print final cost and matrices.

Python Code

```
File Edit Format Run Options Windows Help
keys = [10,12,16,21]
freq = [4,2,6,3]

n = len(freq)

cost = [[0]*(n+1) for _ in range(n+1)]

for i in range(n):
    cost[i][i+1] = freq[i]

for l in range(2,n+1):
    for i in range(n-l+1):
        j = i+l
        cost[i][j] = 999
        s = sum(freq[i:j])

        for r in range(i+1,j+1):
            c = cost[i][r-1] + cost[r][j] + s

            if c < cost[i][j]:
                cost[i][j] = c

print("Minimum Cost =", cost[0][n])
|
```

Input

Keys = {10,12,16,21}

Frequencies = {4,2,6,3}

Output

```
>>> ===== RESTART =====
>>>
('Minimum Cost =', 26)
```

Result

The OBST minimizes total searching cost efficiently.

Dynamic Programming helps compute the optimal tree structure.

17. A game on an undirected graph is played by two players, Mouse and Cat, who alternate turns. The graph is given as follows: $\text{graph}[a]$ is a list of all nodes b such that ab is an edge of the graph. The mouse starts at node 1 and goes first, the cat starts at node 2 and goes second, and there is a hole at node 0. During each player's turn, they must travel along one edge of the graph that meets where they are. For example, if the Mouse is at node 1, it must travel to any node in $\text{graph}[1]$. Additionally, it is not allowed for the Cat to travel to the Hole (node 0). Then, the game can end in three ways:

If ever the Cat occupies the same node as the Mouse, the Cat wins.

If ever the Mouse reaches the Hole, the Mouse wins.

If ever a position is repeated (i.e., the players are in the same position as a previous turn, and it is the same player's turn to move), the game is a draw.

Given a graph, and assuming both players play optimally, return 1 if the mouse wins the game,

2 if the cat wins the game, or

0 if the game is a draw.

Example 1:

Input: $\text{graph} = [[2,5],[3],[0,4,5],[1,4,5],[2,3],[0,2,3]]$

Output: 0

Example 2:

Input: $\text{graph} = [[1,3],[0],[3],[0,2]]$

Output: 1

Aim

To determine whether the Mouse wins, Cat wins, or the game ends in a draw using Dynamic Programming.

Algorithm

1. Start Mouse at node 1 and Cat at node 2.
2. If Mouse reaches node 0 $\rightarrow$ Mouse wins.
3. If Cat reaches Mouse node $\rightarrow$ Cat wins.
4. Mouse and Cat move alternately through graph edges.
5. If the same state repeats $\rightarrow$ Draw.

Python Code

File Edit Format Run Options Windows Help

```
graph = [[2,5],[3],[0,4,5],[1,4,5],[2,3],[0,2,3]]
```

```
def solve(mouse, cat, turn):

    if mouse == 0:
        return 1

    if mouse == cat:
        return 2

    if turn == 15:
        return 0

    if turn % 2 == 0: # Mouse move
        ans = 2
        for x in graph[mouse]:
            r = solve(x, cat, turn+1)

            if r == 1:
                return 1

            if r == 0:
                ans = 0

        return ans

    else: # Cat move
        ans = 1
        for x in graph[cat]:

            if x != 0:
                r = solve(mouse, x, turn+1)

                if r == 2:
                    return 2

                if r == 0:
                    ans = 0

        return ans

print(solve(1,2,0))
```


Input

graph = [[2,5],[3],[0,4,5],[1,4,5],[2,3],[0,2,3]]

Output

```
>>> ===== RESTART =====  
>>>  
0
```

Result

The program simulates optimal moves of Mouse and Cat.

Output 0 indicates the game ends in a draw.

18. You are given an undirected weighted graph of n nodes (0-indexed), represented by an edge list where $\text{edges}[i] = [a, b]$ is an undirected edge connecting the nodes a and b with a probability of success of traversing that edge $\text{succProb}[i]$. Given two nodes start and end , find the path with the maximum probability of success to go from start to end and return its success probability. If there is no path from start to end , return 0. Your answer will be accepted if it differs from the correct answer by at most $1e-5$.

Example 1:

Input: $n = 3$, $\text{edges} = [[0,1],[1,2],[0,2]]$, $\text{succProb} = [0.5,0.5,0.2]$, $\text{start} = 0$, $\text{end} = 2$

Output: 0.25000

Explanation: There are two paths from start to end , one having a probability of success = 0.2 and the other has $0.5 * 0.5 = 0.25$.

Example 2:

Input: $n = 3$, $\text{edges} = [[0,1],[1,2],[0,2]]$, $\text{succProb} = [0.5,0.5,0.3]$, $\text{start} = 0$, $\text{end} = 2$

Output: 0.30000

Aim

To find the path with maximum success probability between two nodes.

Algorithm

1. Create graph with probabilities.
2. Use Priority Queue (Max Heap).
3. Always choose path with maximum probability.
4. Update probabilities for neighboring nodes.

Python Code

```
import heapq

def maxProbability(n, edges, succProb, start, end):

    graph = [[] for _ in range(n)]

    for (u,v),p in zip(edges,succProb):
        graph[u].append((v,p))
        graph[v].append((u,p))

    pq = [(-1,start)]
    prob = [0]*n
    prob[start] = 1

    while pq:
        p,node = heapq.heappop(pq)
        p = -p

        if node == end:
            return p

        for nei,np in graph[node]:
            newp = p*np

            if newp > prob[nei]:
                prob[nei] = newp
                heapq.heappush(pq, (-newp,nei))

    return 0

n = 3
edges = [[0,1],[1,2],[0,2]]
succProb = [0.5,0.5,0.2]

print(maxProbability(n,edges,succProb,0,2))
```

Input

```
n = 3
edges = [[0,1],[1,2],[0,2]]
succProb = [0.5,0.5,0.2]
start = 0, end = 2
```

Output

```
>>> ===== RESTART =====
>>>
0.25
>>>
```

Result

The algorithm finds the path with maximum probability.
The best probability obtained is 0.25.

19. There is a robot on an $m \times n$ grid. The robot is initially located at the top-left corner (i.e., `grid[0][0]`). The robot tries to move to the bottom-right corner (i.e., `grid[m - 1][n - 1]`). The robot can only move either down or right at any point in time. Given the two integers m and n , return the number of possible unique paths that the robot can take to reach the bottom-right corner. The test cases are generated so that the answer will be less than or equal to $2 * 10^9$.

Example 1:

START

FINISH

Input: $m = 3, n = 7$

Output: 28

Example 2:

Input: $m = 3, n = 2$

Output: 3

Explanation: From the top-left corner, there are a total of 3 ways to reach the bottom-right corner:

1. Right -> Down -> Down

2. Down -> Down -> Right

3. Down -> Right -> Down

Aim

To find the number of unique paths from top-left to bottom-right in a grid using Dynamic Programming.

Algorithm

1. Create a DP table of size $m \times n$.
2. Fill first row and first column with 1.
3. For remaining cells:
 $dp[i][j] = dp[i-1][j] + dp[i][j-1]$
4. Return last cell value.

Python Code

```
File Edit Format Run Options Windows Help
def uniquePaths(m,n):
    dp = [[1]*n for _ in range(m)]
    for i in range(1,m):
        for j in range(1,n):
            dp[i][j] = dp[i-1][j] + dp[i][j-1]
    return dp[m-1][n-1]
print(uniquePaths(3,7))
print(uniquePaths(3,2))
```

Input

m = 3, n = 7

m = 3, n = 2

Output

```
>>> ----- RESTART -----  
>>>  
28  
3  
>>>
```

Result

The program calculates all possible unique paths in the grid.

Dynamic Programming avoids repeated calculations.

20. Given an array of integers nums, return the number of good pairs. A pair (i, j) is called good if $\text{nums}[i] == \text{nums}[j]$ and $i < j$.

Example 1:

Input: nums = [1,2,3,1,1,3]

Output: 4

Explanation: There are 4 good pairs (0,3), (0,4), (3,4), (2,5) 0-indexed.

Example 2:

Input: nums = [1,1,1,1]

Output: 6

Explanation: Each pair in the array are good.

Aim

To count the number of good pairs in an array where $\text{nums}[i] = \text{nums}[j]$ and $i < j$.

Algorithm

1. Traverse array elements.
2. Count frequency of each number.
3. For every repeated number, add count to answer.
4. Return total pairs.

Python Code

```
File Edit Format Run Options Windows Help
def goodPairs(nums):

    d = {}
    ans = 0

    for x in nums:

        if x in d:
            ans += d[x]
            d[x] += 1

        else:
            d[x] = 1

    return ans

print(goodPairs([1,2,3,1,1,3]))
print(goodPairs([1,1,1,1]))
```

Input

```
nums = [1,2,3,1,1,3]
nums = [1,1,1,1]
```

Output

```
>>> ===== RESTART =====
>>>
4
6
.
```

Result

The program counts all valid good pairs efficiently.
Time complexity is $O(n)$.

21. There are n cities numbered from 0 to $n-1$. Given the array `edges` where `edges[i] = [fromi, toi, weighti]` represents a bidirectional and weighted edge between cities `fromi` and `toi`, and given the integer `distanceThreshold`. Return the city with the smallest number of cities that are reachable through some path and whose distance is at most `distanceThreshold`. If there are multiple such cities, return the city with the greatest number. Notice that the distance of a path connecting cities i and j is equal to the sum of the edges' weights along that path.

Example 1:

Input: $n = 4$, `edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]]`, `distanceThreshold = 4`

Output: 3

Explanation: The figure above describes the graph.

The neighboring cities at a `distanceThreshold = 4` for each city are: City

0 -> [City 1, City 2]

City 1 -> [City 0, City 2, City 3]

City 2 -> [City 0, City 1, City 3]

City 3 -> [City 1, City 2]

Cities 0 and 3 have 2 neighboring cities at a `distanceThreshold = 4`, but we have to return city 3 since it has the greatest number.

Example 2:

Input: $n = 5$, `edges = [[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]]`, `distanceThreshold = 2`

Output: 0

Explanation: The figure above describes the graph.

The neighboring cities at a `distanceThreshold = 2` for each city are: City

0 -> [City 1]

City 1 -> [City 0, City 4]

City 2 -> [City 3, City 4]

City 3 -> [City 2, City 4]

City 4 -> [City 1, City 2, City 3]

The city 0 has 1 neighboring city at a `distanceThreshold = 2`.

Aim

To find the city that can reach the smallest number of cities within the given distance threshold using Floyd Warshall Algorithm.

Algorithm

1. Create distance matrix.
2. Initialize edge weights.
3. Apply Floyd Warshall Algorithm.
4. Count reachable cities for each city.
5. Return city with minimum reachable cities.

Python Code

File Edit Format Run Options Windows Help

```
INF = 99999
```

```
def findCity(n, edges, threshold):

    dist = [[INF]*n for _ in range(n)]

    for i in range(n):
        dist[i][i] = 0

    for u,v,w in edges:
        dist[u][v] = w
        dist[v][u] = w

    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(dist[i][j],
                                   dist[i][k] + dist[k][j])

    ans = -1
    mini = INF

    for i in range(n):

        count = 0

        for j in range(n):

            if dist[i][j] <= threshold and i != j:
                count += 1

        if count <= mini:
            mini = count
            ans = i

    return ans

print(findCity(4,
[[0,1,3],[1,2,1],[1,3,4],[2,3,1]],4))

print(findCity(5,
[[0,1,2],[0,4,8],[1,2,3],
[1,4,2],[2,3,1],[3,4,1]],2))
```

Input

n = 4

edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]]

distanceThreshold = 4

n = 5

edges = [[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]]

distanceThreshold = 2

Output

```
>>> ===== RESTART =====  
>>>  
3  
0  
...
```

Result

The program finds the city with minimum reachable neighbors.

Floyd Warshall computes shortest paths between all cities.

22. You are given a network of n nodes, labeled from 1 to n . You are also given `times`, a list of travel times as directed edges `times[i] = (ui, vi, wi)`, where ui is the source node, vi is the target node, and wi is the time it takes for a signal to travel from source to target. We will send a signal from a given node k . Return the minimum time it takes for all the n nodes to receive the signal. If it is impossible for all the n nodes to receive the signal, return -1.

Example 1:

Input: `times = [[2,1,1],[2,3,1],[3,4,1]]`, $n = 4$, $k = 2$

Output: 2

Example 2:

Input: `times = [[1,2,1]]`, $n = 2$, $k = 1$

Output: 1

Example 3:

Input: `times = [[1,2,1]]`, $n = 2$, $k = 2$

Output: -1

Aim

To find the minimum time required for all nodes to receive the signal using Dijkstra's Algorithm.

Algorithm

1. Create graph using adjacency list.
2. Use priority queue for shortest path calculation.
3. Update minimum signal times for neighbors.
4. Return maximum time among all nodes.

Python Code

```
import heapq

def networkDelay(times, n, k):
    graph = {}

    for u,v,w in times:
        if u not in graph:
            graph[u] = []

        graph[u].append((v,w))

    pq = [(0,k)]
    dist = {}

    while pq:
        time,node = heapq.heappop(pq)

        if node in dist:
            continue

        dist[node] = time

        if node in graph:
            for nei,w in graph[node]:
                if nei not in dist:
                    heapq.heappush(pq, (time+w,nei))

    if len(dist) == n:
        return max(dist.values())

    return -1

print(networkDelay([[2,1,1],[2,3,1],[3,4,1]],4,2))
print(networkDelay([[1,2,1]],2,1))
print(networkDelay([[1,2,1]],2,2))
|
```

Input

times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2

times = [[1,2,1]], n = 2, k = 1

times = [[1,2,1]], n = 2, k = 2

Output

```
>>> ===== RESTART =====  
>>>  
2  
1  
-1  
~ ~ ~
```

Result

The program computes minimum signal transmission time efficiently.
Dijkstra's Algorithm finds shortest paths from the source node.